

RAG Notes

Module 1: Introduction to RAG

Fundamentals

What is RAG?

RAG (Retrieval-Augmented Generation) is an AI technique that improves LLM responses by **retrieving relevant information from external data sources before generating an answer.**

Instead of relying only on its training knowledge, the LLM first searches for related documents and then uses them to generate an accurate response.

Simple Flow

User Question
↓
Retriever searches documents
↓
Relevant documents found
↓
LLM reads documents
↓
Generates accurate answer

Why RAG?

LLMs have limited knowledge because they only know what they learned during training.

RAG allows them to use **up-to-date and private information.**

Why we use RAG

- Access latest information
 - Reduce hallucinations
 - Answer using company data
 - Improve accuracy
 - No need to retrain the model
-

Problems with LLMs

Although LLMs are powerful, they have several limitations.

Common Problems

- Hallucinations
 - Knowledge cutoff
 - Cannot access private company data
 - Information becomes outdated
 - Difficult to update knowledge
-

Hallucination

Hallucination occurs when an LLM **generates incorrect or completely made-up information while sounding confident.**

Example

Question

Who won the FIFA World Cup in 2032?

The model may invent a fake winner because that event hasn't happened.

Why it happens

- Missing information
 - Weak context
 - Guessing based on patterns
-

Knowledge Cutoff

An LLM only knows information available **up to its training date.**

It cannot automatically learn new events after training.

Example

A model trained until 2024 cannot know news from 2026 unless external information is provided.

External Knowledge

External knowledge refers to **information stored outside the LLM**.

Examples include:

- PDFs
- Company documents
- Databases
- Websites
- APIs
- Research papers

RAG retrieves this information before generating an answer.

Dynamic Knowledge

Dynamic knowledge is **information that changes frequently**.

Examples

- Stock prices
- Weather
- News
- Sports scores
- Product inventory

RAG can retrieve the latest version of this information.

Parametric Memory

Parametric memory is the knowledge **stored inside the model's parameters (weights)** during training.

Characteristics

- Learned during training
- Cannot be easily updated
- Stored permanently until retraining

Example

Knowing that:

Earth revolves around the Sun.

Non-Parametric Memory

Non-parametric memory is **knowledge stored outside the model** in external storage.

Examples

- Vector databases
- PDFs
- Documents
- Knowledge bases

The model retrieves this information when needed.

RAG vs Fine-Tuning

RAG	Fine-Tuning
Uses external documents	Updates model weights
No retraining required	Requires retraining
Easy to update knowledge	Expensive to update
Good for changing data	Good for learning new behavior
Fast implementation	Time-consuming

Simple Rule

- **Need latest knowledge → Use RAG**
 - **Need new skills or behavior → Fine-Tuning**
-

RAG vs Prompt Engineering

RAG	Prompt Engineering
Retrieves external information	Uses only better prompts
Accesses private documents	Uses model's existing knowledge
Better for factual accuracy	Better for improving instructions
Handles large knowledge bases	Limited by prompt context

Simple Rule

- Prompt Engineering improves **how you ask**.
 - RAG improves **what information the model knows during answering**.
-

RAG vs Long Context Models

RAG	Long Context Models
Retrieves only relevant documents	Reads a very large context directly
More efficient	More computationally expensive
Works well with huge document collections	Best when all needed information fits in the context window
Easier to scale	Limited by maximum context size

Simple Rule

- **Large document collection → RAG**
 - **Single large document → Long Context Model**
-

Advantages of RAG

Benefits

- Reduces hallucinations
 - Uses latest information
 - Works with private data
 - No retraining needed
 - Better factual accuracy
 - Easy to update knowledge
 - Cost-effective compared to retraining
 - Scalable for large knowledge bases
-

Limitations of RAG

Drawbacks

- Depends on retrieval quality
- Poor retrieval leads to poor answers
- Slightly slower because of document search

- Requires maintaining external databases
 - More complex than using an LLM alone
-

Real-world Applications

RAG is widely used wherever AI needs accurate, up-to-date, or private information.

Common Applications

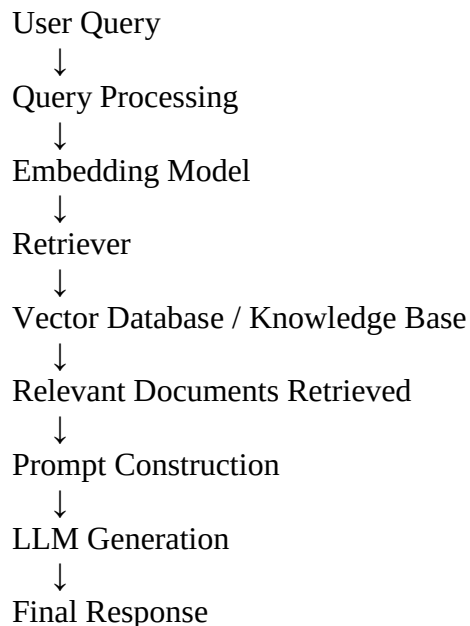
- **Customer Support** – Answer questions using company manuals and FAQs.
- **Enterprise Search** – Search internal documents, policies, and reports.
- **Healthcare** – Retrieve medical guidelines and research papers.
- **Legal AI** – Search contracts, laws, and case documents.
- **Education** – Answer questions from textbooks and lecture notes.
- **Finance** – Retrieve market reports and financial documents.
- **Research Assistants** – Search scientific papers before answering.
- **Chatbots** – Use business-specific knowledge to provide accurate responses.

Module 2: RAG Architecture

Complete Pipeline

RAG works by **retrieving relevant information first** and then using an **LLM to generate the final answer**.

RAG Pipeline



RAG Architecture

RAG architecture combines **information retrieval** with **text generation**.

Instead of answering only from its training knowledge, the LLM first retrieves relevant documents and then generates an answer using those documents.

Main Parts

- User
 - Query Processing
 - Retriever
 - Embedding Model
 - Vector Database
 - Knowledge Base
 - Retrieved Context
 - Prompt Construction
 - LLM
 - Final Response
-

Components of RAG

The RAG system consists of multiple components that work together to generate accurate responses.

User Query

The **User Query** is the question or request given by the user.

It is the starting point of the RAG pipeline.

Example

User asks:

What is Retrieval-Augmented Generation?

Query Processing

Query Processing prepares the user's question before searching.

It may:

- Clean the query
- Remove unnecessary words
- Rewrite the query
- Convert it into an embedding

This helps improve retrieval accuracy.

Retriever

The **Retriever** searches the knowledge base to find the most relevant documents related to the user's query.

It does **not generate answers**.

Its job is only to find useful information.

Example

User asks:

Explain transformers.

Retriever searches documents about transformers and returns the most relevant ones.

Knowledge Base

The **Knowledge Base** is the collection of information that RAG searches.

It stores all documents that the system can retrieve.

Examples

- PDFs
- Word documents
- Text files
- Company documents
- Websites
- Research papers

- Databases
-

Embedding Model

An **Embedding Model** converts text into **numerical vectors (embeddings)**.

These vectors represent the meaning of the text.

Similar texts produce similar vectors.

Example

Text:

Machine Learning

↓

Vector:

[0.42, -0.18, 0.91, ...]

The retriever compares these vectors to find similar documents.

Vector Database

A **Vector Database** stores document embeddings and performs fast similarity searches.

Instead of searching by exact words, it searches by **meaning (semantic similarity)**.

Popular Vector Databases

- Pinecone
 - ChromaDB
 - FAISS
 - Weaviate
 - Milvus
 - Qdrant
-

Retrieved Context

Retrieved Context is the set of documents or text chunks returned by the retriever.

These documents are passed to the LLM as additional knowledge.

Example

User asks:

What is CNN?

Retrieved Context:

- CNN definition
 - CNN architecture
 - CNN applications
-

Prompt Construction

Prompt Construction combines:

- User Query
- Retrieved Context
- Instructions for the LLM

This creates the final prompt that is sent to the LLM.

Example

Context:

CNN is a deep learning model used for image processing.

Question:

What is CNN?

Answer using the given context only.

LLM Generation

The LLM reads the constructed prompt and generates the final answer.

Unlike a normal LLM, it uses both:

- Retrieved documents

- Its own learned knowledge

This improves accuracy and reduces hallucinations.

Final Response

The **Final Response** is the answer returned to the user.

It is generated using:

- User query
- Retrieved context
- LLM reasoning

The goal is to provide an accurate, relevant, and context-aware response.

Module 3: Knowledge Base

Documents

Documents are the **source of information** used in a RAG system.

The knowledge base stores these documents so they can be retrieved when answering user questions.

Examples

- PDFs
 - Word files
 - Web pages
 - CSV files
 - Databases
 - APIs
-

Structured Data

Structured data is **organized in rows and columns** with a fixed format.

It is easy to search and process.

Examples

- SQL databases
 - Excel sheets
 - CSV files
-

Semi-Structured Data

Semi-structured data has some organization but **does not follow a strict table format**.

It uses tags or key-value pairs.

Examples

- JSON
 - XML
 - HTML
-

Unstructured Data

Unstructured data has **no fixed format**.

It usually contains free-form text or media.

Examples

- PDFs
 - Word documents
 - Emails
 - Images
 - Audio
 - Videos
-

PDFs

PDF files are one of the most common knowledge sources in RAG.

They may contain:

- Text
- Tables
- Images

- Charts

The text is extracted before indexing.

Word Documents

Word documents (.doc/.docx) contain formatted text and images.

Their text is extracted and added to the knowledge base.

HTML

HTML is the language used to create web pages.

RAG extracts useful text while ignoring unnecessary tags.

Web Pages

Web pages provide online information that can be indexed into a knowledge base.

Examples include:

- Blogs
 - Documentation
 - News articles
 - FAQs
-

Databases

Databases store structured information that RAG can retrieve.

Examples

- Customer records
 - Product details
 - Employee information
-

CSV Files

CSV (Comma-Separated Values) files store tabular data.

Each row represents a record.

Example

```
Name, Age  
John, 25  
Alice, 30
```

JSON

JSON (JavaScript Object Notation) stores data as **key-value pairs**.

It is widely used in APIs.

Example

```
{  
  "name": "Laptop",  
  "price": 55000  
}
```

APIs

APIs allow RAG to fetch **live or dynamic data** from external systems.

Examples

- Weather API
 - News API
 - Stock Market API
-

Images (Concept)

Images can also be part of a knowledge base.

Since LLMs cannot directly understand images, they are processed using **vision models or OCR** to extract useful information.

Audio (Concept)

Audio files contain spoken information.

Speech-to-text models convert audio into text before storing it in the knowledge base.

Document Processing

Document Processing prepares documents before they are stored in the knowledge base.

Typical steps include:

- Loading
 - Parsing
 - Cleaning
 - Metadata extraction
 - Normalization
 - Deduplication
-

Document Loading

Document Loading is the process of importing documents from different sources into the RAG pipeline.

Sources

- Local files
 - Cloud storage
 - Websites
 - Databases
-

Parsing

Parsing extracts meaningful content from a document.

It separates:

- Text

- Tables
 - Headings
 - Lists
-

Cleaning

Cleaning removes unnecessary information.

Examples include:

- Extra spaces
- HTML tags
- Empty lines
- Special characters
- Duplicate text

This improves retrieval quality.

Metadata Extraction

Metadata is **information about a document**.

Examples

- Title
- Author
- Date
- File name
- Source
- Language

Metadata helps filter and retrieve documents more effectively.

Normalization

Normalization makes documents consistent.

Examples:

- Convert text to lowercase

- Standardize date formats
 - Remove unnecessary punctuation
 - Normalize spacing
-

Deduplication

Deduplication removes duplicate documents or repeated content.

Benefits:

- Saves storage
 - Faster retrieval
 - Prevents repeated answers
-

Module 4: Text Chunking

Chunking

Chunking is the process of **splitting large documents into smaller pieces (chunks)**.

These chunks are stored separately so the retriever can find the most relevant information.

Why Chunking?

LLMs and vector databases work better with smaller pieces of text.

Chunking helps:

- Improve retrieval accuracy
 - Reduce processing time
 - Fit within the LLM's context window
 - Retrieve only relevant information
-

Chunk Size

Chunk size is the **amount of text in one chunk**.

It can be measured by:

- Characters
- Words
- Sentences
- Tokens

Example

- Small chunk: 100 words
 - Large chunk: 500 words
-

Chunk Overlap

Chunk overlap means **sharing some text between consecutive chunks**.

This preserves context across chunk boundaries.

Example

Chunk 1:
AI is transforming healthcare...

Chunk 2:
...transforming healthcare by improving diagnosis.

The overlapping words help maintain continuity.

Chunking Strategies

Chunking strategies define **how documents are divided** into chunks.

Different strategies suit different types of content.

Fixed-size Chunking

Splits text into equal-sized chunks.

Advantages

- Simple
- Fast

- Easy to implement

Disadvantages

- May split sentences or ideas in the middle
-

Recursive Chunking

Splits text using a hierarchy.

Typical order:

1. Paragraph
2. Sentence
3. Word

It keeps splitting until the chunk reaches the desired size.

Sentence Chunking

Each chunk contains one or more complete sentences.

Advantages

- Preserves sentence meaning
 - Easy to understand
-

Paragraph Chunking

Each paragraph becomes a separate chunk.

Advantages

- Maintains complete ideas
 - Good for articles and reports
-

Semantic Chunking

Splits text based on **meaning**, not size.

Each chunk contains related concepts.

Advantages

- Better retrieval quality
- Preserves context

Disadvantages

- More computationally expensive
-

Sliding Window Chunking

Creates overlapping chunks using a moving window.

Example

Chunk 1 → Sentence 1–5

Chunk 2 → Sentence 4–8

Chunk 3 → Sentence 7–11

This helps preserve context across chunks.

Token-based Chunking

Splits text based on the **number of tokens** instead of words or characters.

This is commonly used because LLMs process tokens.

Document-aware Chunking

Splits documents while respecting their structure.

Examples:

- Headings

- Sections
- Tables
- Lists

This keeps related information together.

Trade-offs

There is **no perfect chunk size or strategy**.

Choosing one involves balancing retrieval quality, context, and efficiency.

Large vs Small Chunks

Small Chunks	Large Chunks
More precise retrieval	More context
Faster search	Better understanding
Less context per chunk	Slower retrieval
May miss related information	May include irrelevant information

Recall vs Precision

Recall

Recall measures **how many relevant chunks are retrieved**.

- Higher recall → More relevant information found.

Precision

Precision measures **how many retrieved chunks are actually relevant**.

- Higher precision → Fewer irrelevant chunks returned.

A good chunking strategy balances both.

Chunk Granularity

Granularity refers to **how detailed or coarse the chunks are**.

- **Fine granularity** → Small chunks with detailed information.
- **Coarse granularity** → Large chunks containing more context.

Choosing the right granularity depends on the application and document type.

Module 5: Embeddings

Embedding Fundamentals

Embeddings are the foundation of semantic search in RAG.

They convert text into **numerical vectors** so that computers can understand the meaning of text.

Embeddings

An **embedding** is a numerical representation (vector) of text.

Texts with similar meanings have similar embeddings.

Example

"Artificial Intelligence"

↓

[0.32, -0.18, 0.74, ...]

The embedding captures the meaning, not just the words.

Dense Embeddings

Dense embeddings are vectors where **most values are non-zero**.

They capture semantic meaning and are widely used in modern RAG systems.

Characteristics

- Compact representation
 - Captures meaning
 - Used by most embedding models
-

Sparse Embeddings

Sparse embeddings are vectors where **most values are zero**.

They mainly represent keyword-based information.

Characteristics

- Many zero values
 - Good for keyword matching
 - Often used in traditional search
-

Embedding Space

Embedding space is the **multi-dimensional space** where embeddings are stored.

Texts with similar meanings are placed close together.

Example

Cat •

Dog •

Car •

"Cat" and "Dog" are closer because they are semantically similar.

Similarity

Similarity measures **how close two embeddings are**.

Higher similarity means the texts have similar meanings.

Similarity is used to retrieve relevant documents.

Cosine Similarity

Cosine Similarity measures the **angle between two vectors**.

Values range from **-1 to 1**.

- **1** → Exactly similar
- **0** → Unrelated
- **-1** → Opposite

It is the **most commonly used** similarity measure in RAG.

Euclidean Distance

Euclidean Distance measures the **straight-line distance** between two vectors.

- Smaller distance → More similar
 - Larger distance → Less similar
-

Dot Product

Dot Product measures similarity by multiplying corresponding vector values.

Higher dot product generally indicates greater similarity.

Some embedding models are optimized for dot product search.

Embedding Models

Embedding models convert text into embeddings.

Different models provide different levels of quality, speed, and multilingual support.

Sentence Embeddings

Sentence embeddings represent the **meaning of an entire sentence** as a single vector.

Example

Sentence:

AI is transforming healthcare.



One embedding vector representing the whole sentence.

OpenAI Embeddings

OpenAI embedding models generate **high-quality embeddings** for semantic search and RAG.

Features

- High accuracy
 - Strong semantic understanding
 - Supports many languages
-

BGE Embeddings

BGE (BAAI General Embedding) is an open-source embedding model.

Features

- High retrieval performance
 - Popular for RAG
 - Good semantic search quality
-

E5 Embeddings

E5 (Embedding from Text-to-Text) is an embedding model designed for retrieval tasks.

Features

- Optimized for search
 - Strong retrieval accuracy
 - Open source
-

Instructor Embeddings

Instructor embeddings create embeddings based on **task instructions**.

They adapt the embedding according to the task.

Example

Instruction:

Represent this document for retrieval.

Multilingual Embeddings

Multilingual embeddings support **multiple languages**.

Documents and queries in different languages can still be matched based on meaning.

Example

English:

Hello

Spanish:

Hola

Both have similar embeddings.

Embedding Considerations

Choosing an embedding model depends on:

- Accuracy
 - Speed
 - Cost
 - Language support
 - Domain
-

Embedding Dimensions

Embedding dimensions represent the **length of the embedding vector**.

Example

- 384 dimensions
- 768 dimensions
- 1024 dimensions
- 1536 dimensions

Higher dimensions often capture more information but require more storage and computation.

Embedding Quality

Embedding quality determines **how well similar meanings are grouped together**.

High-quality embeddings improve retrieval accuracy.

Good embeddings produce better RAG responses.

Domain-specific Embeddings

Domain-specific embeddings are trained for a particular field.

Examples

- Medical
- Legal
- Finance
- Scientific research

They better understand specialized terminology.

Embedding Drift

Embedding drift occurs when **stored embeddings become outdated**.

Reasons include:

- New documents
- Updated embedding models
- Changes in language or data

Re-embedding the data helps maintain retrieval quality.

Module 6: Vector Databases

Fundamentals

A vector database stores **embeddings (vectors)** and allows fast similarity searches.

Instead of searching by exact words, it searches by **meaning**.

Vector Database

A **Vector Database** is a database designed to store and search vector embeddings efficiently.

It is a core component of RAG systems.

Vector Index

A **Vector Index** is a data structure that speeds up similarity searches.

Instead of comparing every vector, it quickly finds the nearest ones.

Benefits

- Faster search
 - Better scalability
 - Lower response time
-

Metadata

Metadata is **additional information about stored vectors**.

It helps filter search results.

Examples

- Title
- Author

- Source
 - Date
 - Category
 - Language
-

IDs

Each stored vector has a unique **ID**.

The ID is used to identify, update, or delete the vector.

Example

ID: doc_001

Title: RAG Notes

Operations

Vector databases support common operations for managing vectors.

Insert

Insert adds a **new vector** to the database.

Example

Add a new document embedding.

Update

Update modifies an **existing vector or its metadata**.

Example

Correct the title or replace the embedding.

Delete

Delete removes a vector from the database.

Example

Remove outdated documents.

Upsert

Upsert means:

- **Update** if the vector already exists.
- **Insert** if it does not exist.

This avoids duplicate records.

Search

Search finds vectors that are **most similar** to the query embedding.

The result is a ranked list of relevant documents.

Filtering

Filtering limits search results using metadata.

Examples

- Only English documents
- Only PDFs
- Only documents after 2025
- Only Finance category

Filtering improves search relevance.

Popular Vector Databases

Several vector databases are widely used in RAG applications.

FAISS

FAISS (Facebook AI Similarity Search) is an open-source library for fast vector similarity search.

Features

- Very fast
 - Runs locally
 - Popular for research and prototypes
-

ChromaDB

ChromaDB is an open-source vector database built specifically for AI applications.

Features

- Easy to use
 - Lightweight
 - Popular for RAG projects
-

Pinecone

Pinecone is a fully managed cloud vector database.

Features

- No infrastructure management
 - Highly scalable
 - Suitable for production systems
-

Weaviate

Weaviate is an open-source vector database with built-in AI features.

Features

- Semantic search

- Hybrid search
 - GraphQL support
-

Milvus

Milvus is an open-source vector database designed for large-scale applications.

Features

- High performance
 - Distributed architecture
 - Handles billions of vectors
-

Qdrant

Qdrant is an open-source vector database optimized for semantic search.

Features

- Fast filtering
 - High performance
 - Easy API integration
-

pgvector

pgvector is a PostgreSQL extension that adds vector search capabilities.

Features

- Uses PostgreSQL
- Supports vector similarity search
- Good when relational and vector data are stored together

Module 7: Vector Search

Similarity Search

Similarity Search finds **documents whose embeddings are most similar** to the query embedding.

Instead of matching exact words, it searches by **meaning (semantic similarity)**.

Example

Query:

"How does RAG work?"

The search may retrieve documents containing "**Retrieval-Augmented Generation**", even if the exact words differ.

Nearest Neighbor Search

Nearest Neighbor Search finds the **closest vectors** to the query vector in the embedding space.

The closest vectors represent the most relevant documents.

Exact Search

Exact Search compares the query vector with **every vector** in the database.

Advantages

- Very accurate
- Finds the true nearest neighbors

Disadvantages

- Slow for large datasets
 - High computational cost
-

Approximate Nearest Neighbor (ANN)

ANN finds vectors that are **very close to the nearest neighbors** without checking every vector.

It sacrifices a little accuracy for much faster search.

Advantages

- Very fast

- Scales to millions of vectors
 - Widely used in production RAG systems
-

ANN Algorithms

ANN algorithms are techniques that speed up similarity search.

Common algorithms include:

- HNSW
 - IVF
 - PQ
 - ScaNN
-

HNSW

HNSW (Hierarchical Navigable Small World) is one of the most popular ANN algorithms.

It organizes vectors into multiple graph layers for fast search.

Features

- High accuracy
 - Fast retrieval
 - Widely used in vector databases
-

IVF

IVF (Inverted File Index) groups similar vectors into clusters.

During search, only the most relevant clusters are searched.

Benefits

- Faster than exact search
 - Good for large datasets
-

PQ (Concept)

PQ (Product Quantization) compresses vectors into smaller representations.

Benefits

- Reduces memory usage
 - Speeds up search
 - Slightly reduces accuracy
-

ScaNN (Concept)

ScaNN (Scalable Nearest Neighbors) is a fast ANN search algorithm developed by Google.

Features

- Optimized for large datasets
 - High search speed
 - Good accuracy
-

Search Parameters

Search parameters control **how the vector search is performed**.

Common parameters include:

- Top-K Retrieval
 - Score Threshold
 - Distance Metric
 - Metadata Filtering
-

Top-K Retrieval

Top-K Retrieval returns the **K most similar documents**.

Example

- Top-3 → Return the best 3 documents.
 - Top-10 → Return the best 10 documents.
-

Score Threshold

A Score Threshold sets the **minimum similarity score** required for a document to be returned.

Example

Threshold = **0.80**

- Score = **0.92** → Returned
- Score = **0.65** → Ignored

This helps remove less relevant results.

Distance Metrics

Distance metrics measure **how similar or different two vectors are**.

Common metrics:

- Cosine Similarity
- Euclidean Distance
- Dot Product

The choice depends on the embedding model and use case.

Metadata Filtering

Metadata Filtering narrows search results using document metadata.

Examples

- Language = English
- Category = AI
- Year = 2026
- File Type = PDF

This improves retrieval relevance.

Module 8: Retrieval

Retrieval Basics

Retrieval is the process of **finding relevant documents** from the knowledge base before the LLM generates an answer.

It is one of the most important steps in RAG.

Retriever

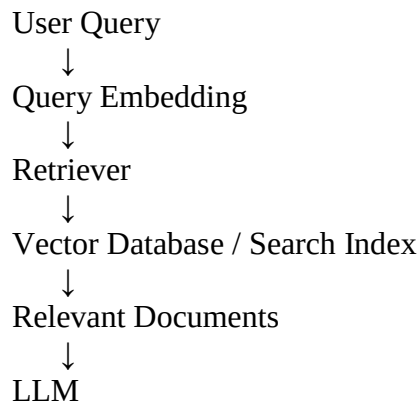
A **Retriever** is the component that searches the knowledge base and returns the most relevant documents.

It retrieves information but **does not generate answers**.

Retrieval Pipeline

The Retrieval Pipeline is the sequence of steps used to retrieve relevant documents.

Pipeline



Candidate Retrieval

Candidate Retrieval is the first stage of retrieval.

The system retrieves a **small set of potentially relevant documents**, which can later be ranked or filtered.

Example

Database contains **100,000 documents**



Retriever selects the **top 50 candidates**



Best documents are passed to the LLM.

Retrieval Methods

Different retrieval methods use different techniques to find relevant documents.

Common methods:

- Dense Retrieval
 - Sparse Retrieval
 - Hybrid Retrieval
 - Lexical Retrieval
 - Semantic Retrieval
 - Traditional Retrieval
-

Dense Retrieval

Dense Retrieval uses **dense embeddings** to search by meaning.

Features

- Semantic understanding
 - Finds similar concepts
 - Used in modern RAG systems
-

Sparse Retrieval

Sparse Retrieval uses **keyword-based vectors** where most values are zero.

Features

- Exact keyword matching

- Faster for specific terms
 - Used in traditional search
-

Hybrid Retrieval

Hybrid Retrieval combines **Dense Retrieval and Sparse Retrieval**.

This provides both semantic understanding and keyword matching.

Benefits

- Better accuracy
 - Higher recall
 - More reliable retrieval
-

Lexical Retrieval

Lexical Retrieval searches documents using **exact words or keywords**.

Example

Query:

"Neural Networks"

Documents containing these exact words are ranked higher.

Semantic Retrieval

Semantic Retrieval searches by **meaning**, not exact words.

Example

Query:

"How do self-driving cars work?"

It may retrieve documents about **autonomous vehicles**, even if the exact phrase is absent.

Traditional Retrieval

Traditional Retrieval relies on **keyword matching and statistical methods** rather than embeddings.

Common methods:

- TF-IDF
 - BM25
-

TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) ranks documents based on the importance of words.

Idea

- Frequently used words in one document are important.
 - Words common across many documents are less important.
-

BM25

BM25 is an improved ranking algorithm based on TF-IDF.

It considers:

- Word frequency
- Document length
- Word importance

BM25 is widely used in search engines.

Hybrid Search

Hybrid Search combines **semantic search** and **keyword search**.

Combines

- Dense Retrieval
- Sparse Retrieval

- BM25
- Embeddings

This often produces better search results than using only one method.

Dense + Sparse Search

Dense + Sparse Search performs both searches simultaneously.

Dense Search

Finds documents by **meaning**.

Sparse Search

Finds documents by **keywords**.

The results are then combined into a final ranked list.

Score Fusion

Score Fusion combines the scores from multiple retrieval methods into a single score.

Example

- Dense score = 0.82
- Sparse score = 0.76

↓

Combined score = Higher confidence ranking

This improves overall retrieval quality.

Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF) is a ranking method that combines results from multiple retrieval systems.

Instead of using similarity scores, it combines the **rank positions** of documents.

Benefits

- Simple and effective
- Works well for hybrid search
- Improves ranking quality

Module 9: Query Processing

Query Understanding

Query Understanding is the process of **analyzing the user's question** to determine its intent and meaning before retrieval.

Example

Query:

"Explain RAG."

The system understands that the user wants an explanation of Retrieval-Augmented Generation.

Query Cleaning

Query Cleaning removes unnecessary words or symbols from the user's query.

Examples

- Extra spaces
- Special characters
- Typographical errors
- Unnecessary punctuation

This improves search quality.

Query Normalization

Query Normalization converts the query into a **consistent format**.

Examples

- Convert text to lowercase
- Standardize dates

- Remove extra spaces
- Expand abbreviations

Example:

AI → Artificial Intelligence

Query Expansion

Query Expansion adds **related words or synonyms** to the query.

This helps retrieve more relevant documents.

Example

Query:

"Car"

Expanded query:

"Car, Automobile, Vehicle"

Query Reformulation

Query Reformulation changes the query into a **clearer or more searchable form** without changing its meaning.

Example

Original:

"Tell me about AI."

Reformulated:

"What is Artificial Intelligence?"

Query Rewriting

Query Rewriting modifies the query to improve retrieval performance.

Unlike reformulation, it focuses on **better search results**.

Example

Original:

"Latest AI"

Rewritten:

"Latest developments in Artificial Intelligence"

Multi-query Generation

Multi-query Generation creates **multiple versions of the same query**.

Each version searches from a different perspective.

Example

Original Query:

"Benefits of RAG"

Generated Queries:

- Advantages of RAG
- Why use RAG?
- Benefits of Retrieval-Augmented Generation

This increases retrieval coverage.

Advanced Query Techniques

Advanced techniques improve retrieval accuracy for complex questions.

Examples include:

- HyDE
- Self-Query Retrieval

- Query Decomposition
 - Step-back Prompting
-

HyDE (Hypothetical Document Embeddings)

HyDE generates a **hypothetical answer** to the user's query first.

The embedding of this generated answer is then used for retrieval instead of the original query.

Benefits

- Better semantic retrieval
 - Improves results for vague queries
-

Self-Query Retrieval

Self-Query Retrieval allows the LLM to generate **both a search query and metadata filters**.

Example

User asks:

AI books after 2023

Generated Search:

- Topic = AI
- Year > 2023

This improves filtered retrieval.

Query Decomposition

Query Decomposition breaks a **complex question into smaller sub-questions**.

Each sub-question is retrieved separately.

Example

Question:

Explain RAG and compare it with Fine-Tuning.

Sub-questions:

- What is RAG?
 - What is Fine-Tuning?
 - How are they different?
-

Step-back Prompting

Step-back Prompting first asks a **broader or higher-level question**, then answers the original question.

This helps improve reasoning and retrieval.

Example

Original:

How does RAG reduce hallucinations?

Step-back:

What is the purpose of RAG?

↓

Answer the original question using that broader understanding.

Module 10: Re-ranking

Re-ranking

Re-ranking is the process of **reordering retrieved documents** based on their relevance to the user's query.

It improves the quality of the documents sent to the LLM.

Why Re-ranking?

Initial retrieval may return some less relevant documents.

Re-ranking selects the **best and most relevant** ones.

Benefits

- Better accuracy
 - Better context
 - Fewer irrelevant documents
 - Improved LLM responses
-

Cross Encoder

A **Cross Encoder** evaluates the **query and document together** to calculate relevance.

Features

- Very accurate
 - Better ranking
 - Slower than bi-encoders
-

Bi-Encoder

A **Bi-Encoder** encodes the query and document **separately** into embeddings.

Similarity is calculated between the two embeddings.

Features

- Fast
 - Efficient
 - Slightly less accurate than cross encoders
-

Candidate Ranking

Candidate Ranking sorts retrieved documents from **most relevant to least relevant**.

Only the top-ranked documents are passed to the LLM.

Relevance Scoring

Each retrieved document receives a **relevance score**.

Higher score → More relevant document.

The scores determine the ranking order.

Context Selection

Context Selection chooses the **best documents or chunks** to include in the final prompt.

The goal is to maximize useful information while staying within the LLM's context limit.

Module 11: Context Construction

Context Building

Context Building is the process of **preparing the retrieved information** before sending it to the LLM.

The goal is to provide the most relevant and useful context.

Context Window

A **Context Window** is the maximum amount of text (tokens) an LLM can process at one time.

If the context is too large, it must be reduced.

Context Packing

Context Packing arranges multiple retrieved chunks **efficiently** within the available context window.

The aim is to include as much useful information as possible.

Context Compression

Context Compression reduces the size of retrieved documents while preserving important information.

This allows more relevant content to fit into the context window.

Context Ordering

Context Ordering arranges retrieved chunks in a logical sequence.

Common strategies:

- Highest relevance first
- Chronological order
- Document order

Well-ordered context helps the LLM generate better answers.

Duplicate Removal

Duplicate Removal eliminates repeated or highly similar chunks.

Benefits

- Saves context space
 - Reduces redundancy
 - Improves response quality
-

Token Budget Management

LLMs have a limited number of tokens they can process.

Token Budget Management ensures the total context stays within this limit while keeping the most valuable information.

Compression

Compression shortens the retrieved context without losing essential meaning.

It helps maximize the use of the context window.

Extractive Compression

Extractive Compression keeps **important sentences exactly as they appear** in the original text.

No new wording is created.

Example

Original:

RAG retrieves external documents before generating answers.

Compressed:

RAG retrieves documents before generating answers.

Abstractive Compression

Abstractive Compression rewrites the content into a **shorter summary** using new wording.

Example

Original:

RAG first retrieves relevant information from external sources and then generates an answer based on that information.

Compressed:

RAG answers questions using retrieved external knowledge.

Context Summarization

Context Summarization creates a **short summary** of the retrieved documents.

This summary is then passed to the LLM instead of the full documents.

Benefits

- Saves tokens
- Keeps key information
- Improves efficiency

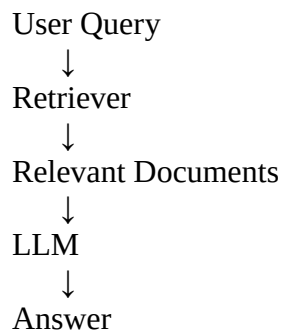
Module 13: Types of RAG

Basic RAG

Basic RAG is the **simplest form of RAG**.

It retrieves relevant documents from a knowledge base and sends them to the LLM to generate an answer.

Flow



Naive RAG

Naive RAG is the **first and simplest implementation** of RAG.

It performs only three steps:

- Retrieve documents
- Add them to the prompt
- Generate the answer

Limitations

- No query optimization
 - No re-ranking
 - No validation
 - May retrieve irrelevant documents
-

Improved RAG

Improved RAG enhances Basic RAG by improving the retrieval process.

Improvements

- Better chunking
- Better embeddings
- Better retrieval
- Metadata filtering
- Re-ranking

Result: More accurate responses than Naive RAG.

Advanced RAG

Advanced RAG includes multiple optimization techniques to improve retrieval and generation.

Features

- Query rewriting
- Re-ranking
- Context compression
- Multi-step retrieval
- Response validation

It is commonly used in production AI systems.

Modular RAG

Modular RAG divides the RAG pipeline into **independent modules**.

Each module can be upgraded or replaced without affecting the others.

Example Modules

- Query Processing
- Retriever
- Re-ranker
- LLM
- Validator

Benefits

- Flexible
 - Easy to maintain
 - Easy to improve individual components
-

Retrieval Variants

Retrieval variants are **different approaches** used to improve document retrieval for various use cases.

Examples:

- Multi-hop RAG
 - Graph RAG
 - Agentic RAG
 - Adaptive RAG
 - CRAG
 - Self-RAG
 - Fusion RAG
-

Multi-hop RAG

Multi-hop RAG retrieves information through **multiple retrieval steps**.

It is useful when one document is not enough to answer the question.

Example

Question:

Which company developed GPT, and who is its CEO?

Step 1:

- Retrieve information about GPT.

Step 2:

- Retrieve information about the company and its CEO.
-

Graph RAG

Graph RAG stores knowledge as a **graph of connected entities and relationships** instead of only text chunks.

Example

```
graph TD
    OpenAI -- developed --> GPT
```

Benefits

- Better reasoning
 - Handles relationships between entities
 - Useful for complex knowledge bases
-

Agentic RAG

Agentic RAG uses an **AI agent** to decide how retrieval should be performed.

The agent can:

- Search multiple sources
- Decide retrieval steps
- Use tools or APIs
- Refine queries automatically

This makes the system more autonomous.

Adaptive RAG

Adaptive RAG changes its retrieval strategy based on the user's query.

Example

- Simple query → Retrieve one document.
- Complex query → Retrieve multiple documents and perform re-ranking.

This improves efficiency and accuracy.

Corrective RAG (CRAG)

CRAG checks the quality of retrieved documents before generating the answer.

If retrieval quality is poor, it performs additional retrieval or correction.

Benefits

- Reduces incorrect context
 - Improves answer quality
 - Reduces hallucinations
-

Self-RAG

Self-RAG allows the LLM to **evaluate its own responses**.

The model decides whether:

- More retrieval is needed
- The answer is sufficient
- The retrieved context is relevant

This improves answer reliability.

Fusion RAG

Fusion RAG combines results from **multiple retrieval methods or multiple queries** before generating the final answer.

Example

Combine:

- Dense Retrieval
- Sparse Retrieval
- Multi-query Retrieval

The merged results provide better context.

Module 14: RAG Optimization

Retrieval Optimization

Retrieval Optimization improves the process of finding the **most relevant documents**.

Better retrieval leads to more accurate answers.

Common improvements include:

- Better Chunking
 - Better Embeddings
 - Better Search
 - Better Re-ranking
-

Better Chunking

Good chunking keeps related information together while maintaining an appropriate chunk size.

Benefits

- Higher retrieval accuracy
 - Better context
 - Less irrelevant information
-

Better Embeddings

Using high-quality embedding models produces better semantic representations of text.

Benefits

- Better similarity search
 - More relevant documents
 - Improved RAG performance
-

Better Search

Better search uses advanced retrieval techniques to improve document selection.

Examples

- Hybrid Search
 - ANN Search
 - Metadata Filtering
 - Query Expansion
-

Better Re-ranking

Re-ranking improves the order of retrieved documents so the **most relevant ones appear first**.

This ensures the LLM receives the best possible context.

Generation Optimization

Generation Optimization improves how the LLM produces the final answer after retrieval.

The focus is on making responses:

- More accurate
 - More relevant
 - More concise
 - Less hallucinatory
-

Prompt Optimization

Prompt Optimization designs better prompts for the LLM.

A good prompt includes:

- User query
- Retrieved context
- Clear instructions

Benefits

- Better responses
 - Fewer hallucinations
 - More consistent outputs
-

Context Compression

Context Compression reduces the size of retrieved information while preserving key details.

Benefits

- Fits within the context window
 - Saves tokens
 - Allows more useful information to be included
-

Response Validation

Response Validation checks whether the generated answer is:

- Correct
- Relevant
- Supported by the retrieved context

If problems are found, the system may retrieve more documents or regenerate the answer.

Performance

Performance refers to the **overall efficiency** of the RAG system.

It is commonly measured using:

- Latency
 - Throughput
 - Accuracy
 - Resource usage
-

Latency

Latency is the **time taken to generate a response** after receiving a user query.

Lower latency provides a faster user experience.

Throughput

Throughput is the **number of queries the system can process in a given time**.

Higher throughput means the system can handle more users simultaneously.

Caching

Caching stores previously computed results so they can be reused.

This avoids repeating expensive operations.

Benefits

- Faster responses
 - Lower cost
 - Reduced computation
-

Embedding Cache

Embedding Cache stores previously generated embeddings.

If the same text appears again, the system reuses the stored embedding instead of generating a new one.

Benefits

- Faster indexing
 - Lower embedding cost
-

Query Cache

Query Cache stores answers or retrieval results for frequently asked queries.

Example

User asks:

What is RAG?

If this query was answered earlier, the cached result can be returned immediately instead of repeating retrieval and generation.

